02-620 Week 8

Machine Learning for Scientists

Aidan Jan

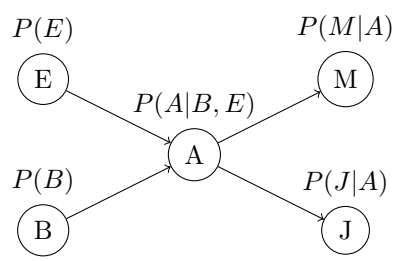
March 16, 2026

Overview: Inference on probability distribution

- Enumeration
- Variable elimination
- Stochastic inference

Enumeration: Joint Distributions

Consider the graph from before:



Recall that $P(B, E, A, J, M) = P(B)P(E)P(A|B, E)P(J|A)P(M|A)$.

Now, suppose we want to find the joint distribution of $P(B, \neg E, A, J, \neg M)$, or in other words,

- "there was a burglary" and "there was not an earthquake" and "alarm went off" and "John called" and "Mary did not call"

while given the probabilities:

- $P(B) = 0.05$
- $P(E) = 0.1$
- $P(A|B, E) = 0.95$, $P(A|B, \neg E) = 0.85$, $P(A|\neg B, E) = 0.5$, $P(A|\neg B, \neg E) = 0.05$
- $P(J|A) = 0.7$, $P(J|\neg A) = 0.05$
- $P(M|A) = 0.8$, $P(M|\neg A) = 0.15$

We can simplify like the following:

$$\begin{aligned}
&P(B, \neg E, A, J, \neg M) \\
&= P(B)P(\neg E)P(A|B, \neg E)P(J|A)P(\neg M|A) \\
&= 0.05 \times 0.9 \times 0.85 \times 0.7 \times 0.2 \\
&= 0.005355
\end{aligned}$$

We can easily compute a **complete joint distribution**. However, what about **partial distributions** and **conditional distributions**?

Partial distribution: $P(B, J, \neg M)$

- Marginalization: $P(X) = \sum_y P(X, Y)$.
- "there was a burglary" and "John called" and "Mary did not call"

We can now simplify:

$$
\begin{aligned}
& P(B, J, \neg M) \\
&= \sum_{a,e} P(B, J, \neg M, A = a, E = e) \\
&= P(B, J, \neg M, A, E) + P(B, J, \neg M, \neg A, E) + P(B, J, \neg M, A, \neg E) + P(B, J, \neg M, \neg A, \neg E) \\
&= 0.0007 + 0.00001 + 0.005 + 0.0003 \\
&= 0.00601
\end{aligned}
$$

Conditional distribution: $P(B|J, \neg M)$

- Conditioning: $P(X|Y) = \frac{P(X,Y)}{P(Y)}$
- Given that "John called" and "Mary did not call", we want to know if "there was a burglary"

We can simplify:

$$
\begin{aligned}
& P(B|J, \neg M) \\
&= \frac{P(B, J, \neg M)}{P(J, \neg M)} \\
&= \frac{\sum_{e,a} P(B, E = e, A = a, J, \neg M)}{\sum_{b,e,a} P(B = b, E = e, A = a, J, \neg M)}
\end{aligned}
$$

Note that in this case:

- The number of possible assignments is exponential in the number of observed variables
- This method can be improved by re-using calculations

Variable Elimination: Inference via Conditional Distribution

We can infer $P(B|J, \neg M)$ from $P(B, J, M, A, E)$ instead of having to calculate all the combinations of variables.

- Query variable: B
- Evidence variables: J, M
- Hidden variables: A, E
- Compute: $P(B|J, \neg M) = \frac{P(B,J,\neg M)}{P(J,\neg M)} = \frac{P(B,J,\neg M)}{P(B,J,\neg M)+P(\neg B,J,\neg M)}$

Now, when we simplify, we get

$$
\begin{aligned}
& P(B, J, \neg M) \\
&= \sum_e \sum_a P(B, J, \neg M, A = a, E = e) \\
&= \sum_e \sum_a P(B)P(e)P(a|B, e)P(J|a)P(\neg M|a)
\end{aligned}
$$

We have 4 terms here, each with 4 multiplications, and three additions. This is a total of 19 operations. We reuse computations rather than recompute probabilities.

$$= P(B) \sum_e P(e) \sum_a P(a|B,e)P(J|a)P(\neg M|a)$$
$$= P(B) \sum_e P(e) f_{A,J,M}(B,e)$$

f is a function of B, E, J, M fixed, a martinalized.

$$= P(B) f_{E,A,J,M}(B)$$

- Now, our inner sum $f_{A,J,M}(B,e) = \sum_a P(a|B,e)P(J|a)P(\neg M|a)$ has two terms times two multiplications each, plus one addition, for a total of 5 operations.
- The next sum $f_{E,A,J,M}(B) = \sum_e P(e) f_{A,J,M}(B,e)$ has 2 terms times (1 + 5 operations for $f_{A,J,M}$), plus 1 addition, for a total of 13 operations
- Finally, we multiply by $P(B)$, which is one more multiplication $\rightarrow$ 14 operations.
- When we reuse the computation, we reduced it from 19 operations to 14!

Conditional Distribution: Normalization

$$P(B|J,\neg M) = \frac{P(B,J,\neg M)}{P(J,\neg M)} = \frac{P(B,J,\neg M)}{P(B,J,\neg M) + P(\neg B,J,\neg M)}$$

- Compute $P(B,J,\neg M)$ and $P(\neg B,J,\neg M)$ (by variable elimination)
- Perform normalization
- Variable elimination can be automatically computed using dynamic programming

Computational Complexity

- We are reusing computation so we are reducing the running time.
- However, there are still cases in which this algorithm will lead to exponential running time.
- Variable elimination can lead to significant cost saving but its efficiency depends on the network structure.
- In a polytree (like our example), no two nodes have more than one path between them.
- For a polytree, there is an algorithm which is linear in the number of nodes.

Estimate Probability from Data Samples

Suppose we can generate samples for B, how to estimate $P(B=1)$?

- We can first generate samples
- Estimator

$$\hat{P}(B=1) = \frac{1}{n} \sum_{i=1}^{n} \mathbb{I}\{b_i = 1\}$$

What about $P(B=1|E=1)$?

- Generate samples: $\{(b_1,e_1),(b_2,e_2),\cdots,(b_n,e_n)\}$

- Estimator:

$$\begin{aligned}\hat{P}(B=1|E=1) &= \frac{\hat{P}(B=1,E=1)}{\hat{P}(E=1)} \\ &= \frac{\frac{1}{n}\sum_{i=1}^{n}\mathbb{I}\{b_i=1,e_i=1\}}{\frac{1}{n}\sum_{i=1}^{n}\mathbb{I}\{e_i=1\}}\end{aligned}$$

Estimate Probability from Data Samples for Bayesian Network

- Generate joint samples across all RVs (B, E, A, J, M)

$$\{(b_i, e_i, a_i, j_i, m_i)\}_{i=1}^{n}$$

- Estimate by counting

$$\begin{aligned}\hat{P}(B|J,\neg M) &= \frac{\hat{P}(B=1,J=1,M=0)}{\hat{P}(J=1,M=0)} \\ &= \frac{\frac{1}{n}\sum_{i=1}^{n}\mathbb{I}\{b_i=1,j_i=1,m_i=0\}}{\frac{1}{n}\sum_{i=1}^{n}\mathbb{I}\{j_i=1,m_i=0\}}\end{aligned}$$

We can't compute the probability of each instance of $(b, e, a, j, m) \in \{0,1\}^5$ and directly sample from the discrete distribution because it will take exponential time.

- Instead, we sample each node sequentially (left to right), according to its conditional distribution given its parents:
  - Start from the top and sample variables with no parents
  - For all other variables: if all parents have been sampled, sample based on conditional distribution
  - In our example, we would sample in the following order:
    1. Sample B from $P(B)$: $(B=1)$
    2. Sample E from $P(E)$: $(E=0)$
    3. Sample A from $P(A|B=1,E=0)$: $(A=1)$
    4. Sample J from $P(J|A=1)$: $(J=1)$
    5. Sample M from $P(M|A=1)$: $(M=0)$

Using the Sampled Data for Inference

- Revisiting our problem: Compute $P(B|J,\neg M)$
- For a large enough N,

$$P(B|J,\neg M) = \frac{P(B,J,\neg M)}{P(J,\neg M)} = \frac{\frac{N_B}{N}}{\frac{N_C}{N}} = \frac{N_B}{N_C}$$

- Looking at the samples we can count:
  - N = total number of samples
  - N_C = total number of samples in which the condition holds $(J,\neg M)$
  - N_B = total number of samples where the joint is true $(B,J,\neg M)$

Overivew: Inference

- Inference on graph
  - Markov Blanket
  - d-separation
- Inference on probability distribution
  - Enumeration: exact method, slow
  - Variable elimination: exact method, faster than enumeration, but still slow
  - Stochastic inference: inexact method, fast

Learning the Model

What to "Learn"

- Probability distribution + graph.

There are two types of learning problems

- Learn **probability distribution** with **known graph structure**
- Learn **probability distribution** and **graph structure**

MLE for Bayesian Network Parameter Estimation

$$\theta \leftarrow \arg\max_{\theta} \log P(\text{data}|\theta)$$

- In our example problem:

$$\theta = \begin{Bmatrix} P(B=1), P(E=1) \\ P(A=1|B=1, E=1), \cdots \\ P(J=1|A=1), P(J=1|A=0) \\ P(M=1|A=1), P(M=1|A=0) \end{Bmatrix}$$

We use MLE to estimate Bayesian network parameters.

$$\log \prod_i P(B_i, E_i, A_i, J_i, M_i) \sum_i \log P(B_i, E_i, A_i, J_i, M_i)$$

This eventually simplifies down to

$$= \sum_i \log P(B_i) + \sum_i \log P(E_i) + \sum_i \log P(A_i|B_i, E_i) + \sum_i \log P(J_i|A_i) + \sum_i \log P(M_i|A_i)$$

- Learning a Bayesian network structure is an **open problem in general!**
- Learning a Bayesian network structure
  - Tree structure (Chow-Liu algorithm) (children only have one parent)
    - Restrictive model structure
    - Efficient learning and inference algorithms
  - A general directed acyclic graph structure
    - Very large search space of candidate BN structures
    - **Inexact method:** heuristic search, efficient
    - **Exact method:** dynamic programming, exponential time complexity

Chow-Liu Algorithm

The Chow-Liu algorithm finds the "best" **tree-structured** network

- Suppose $P(X)$ is the true distribution, $T(X)$ is our tree-structured network, where $X = \langle X_1, \ldots, X_n \rangle$
- Chow-Liu minimizes the Kullback-Leibler (KL) divergence:

$$KL(P(X)||T(X)) \equiv \sum_k P(X = k) \log \frac{P(X = k)}{T(X = k)}$$

- To minimize $KL(P||T)$, it sufficies to find the tree netwrok T that maximizes the sum of mutual information over its edges
- Mutual information for an edge between variable A and B:

$$I(A, B) = \sum_a \sum_b P(a, b) \log \frac{P(a, b)}{P(a)P(b)}$$

The algorithm:

1. For each pair of variables A, B, use data to estimate $P(A, B)$, $P(A)$, and $P(B)$
2. For each pair of variables A, B, calculate mutual information. ($I(A, B)$ using the equation above)
3. Calculate the maximum spanning tree over the set of variables, using edge weights $I(A, B)$. (Given N variables, this costs $O(N^2)$ time)
4. Add arrows to edges to form a DAG by picking an arbitrary node as root and directing edges outward from the root
5. Learn the conditional probability distributions for this graph

MST can be found using Kruskal's Algorithm or Prim's Algorithm.

Hill Climbing Algorithm

This is a heuristic search algorithm.

- Start with an initial structure
- Repeat until termination:
  - Generate a set of structures by modifying the current structure.
    - Add an edge
    - Delete an edge
    - Reverse an edge
    - The add-edge and reverse-edge not permitted if results in a directed cycle
  - Compute their scores
  - Pick the one with the highest score and use it as the current model in the next step
  - Terminate when the model score cannot be improved
- Return the best network

Problems with Hill Climbing

- Local Maxima: All one-edge changes reduced the score, but not optimal yet.
- Plateaus: Neighbors have the same score
- Solutions:
  - Random restart
  - TABU-search:
    - Keep a list of K most recently visited structures and avoid them
    - Avoid plateau
  - Simulated annealing

Bayesian Networks: What you should know

- Representation
  - Bayesian networks represent joint distribution as a DAG + Conditional Distributions
  - Markov blanket, d-separation lets us decode conditional independence assumptions
- Inference
  - NP-hard in general
  - For some graphs, efficient inference is feasible
  - Variable elimination, stochastic methods
- Learning
  - Easy for known graph, fully observed data (MLE's, MAP, etc.)
  - Learning graph structure: Chow-Liu for tree-structured networks
  - Hardest when graph unknown

Graphical Models

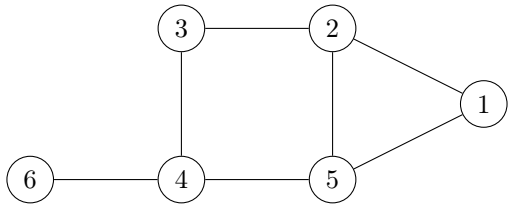
Probabilistic Graphical Models for Undirected Networks

- Bayesian Networks
  - Graph: Directed edges in the network
  - Probability distribution: product of "local" conditional probability distributions for each "node"
- Markov Random Fields
  - Graph: Undirected edges
  - Probability distribution: product of "clique potential functions", normalized to ensure the probability integral is 1

Aside: Cliques

A clique is a subgraph in which all nodes are completely connected. (e.g., every node is connected to every other node in the clique)

Markov Random Fields Satisfy Local Markov Properties



- Pairwise Markov Property: Any two non-adjacent variables are conditionally independent given all other variables
  - E.g., $X_6 \perp X_3 | X_1, X_2, X_4, X_5$
- Local Markov Property: A variable is conditionally independent of all other variables given its neighbors:
  - E.g., $X_6 \perp (X_1, X_2, X_3, X_5) | X_4$
- Global Markov Property: Any two subsets of variables are conditionally independent given a separating subset:
  - E.g., $(X_6, X_4) \perp (X_1, X_2) | X_3, X_5$

Probabilistic Graphical Models for Undirected Networks

- Discrete random variables
  - No restrictions on loop (unlike Bayesian networks requiring no directed cycles)
  - Inference and learning is expensive
- Continuous random variables
  - Gaussian Markov random field
  - Gaussian distribution can be used for this!
  - Called "Gaussian graphical models"

Our focus now will be on continuous random variables.

Gaussian Graphical Models

- Model
  - Review of multivariate Gaussian (normal) distribution
  - Gaussian graphical models
    - Relationship to multivariate Gaussian distribution
- Inference
- Learning a model from data: MLE!

Review: Multivariate Normal Distribution

- Recall the univariate normal distribution for a single random variable x

$$p(x) \sim N(\mu, \sigma^2)$$

- Recall the multivariate normal distribution

$$p(x_1, \ldots, x_8) \sim N(\mu, \Sigma)$$

  where mean vector μ and covariance matrix Σ is:

$$\mu = \begin{bmatrix} \mu_1 \\ \vdots \\ \mu_8 \end{bmatrix} \qquad \Sigma = \begin{bmatrix} \sigma_1^2 & \cdots & \sigma_{18} \\ \vdots & \ddots & \vdots \\ \sigma_{18} & \cdots & \sigma_8^2 \end{bmatrix}$$

- Note that here, $\Sigma^{-1} = \Theta$, called a **precision matrix**. $\Sigma\Theta = I$, where I is the identity matrix.

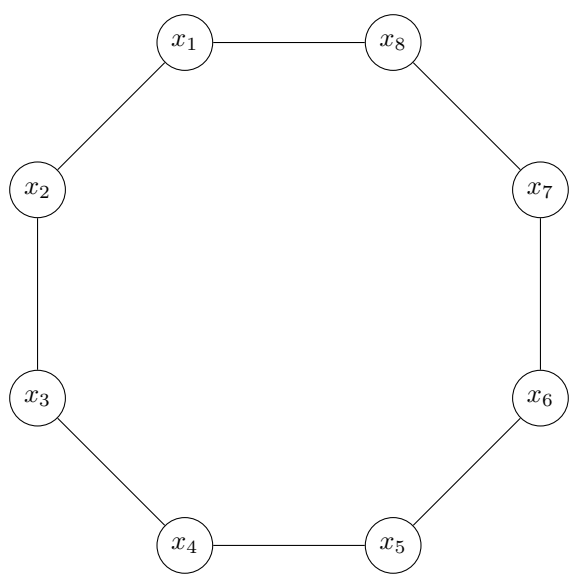
Probabilistic Graph Representation by GGM

A Gaussian Graphical Model uses multivariate normal distribution to represent a probabilistic graph.

- Multivariate normal distribution $p(x_1, \cdots, x_8) \sim N(\mu, Sigma)$
- Precision matrix $\Theta = \Sigma^{-1}$
- $\Theta_{ij} \neq 0$ if and only if there is an edge between x_i and x_j
- This defines a Markov random field (satisfying local Markov properties)

Example: Gaussian Graphical Models

Consider the following GGM, undirected graph:



If we draw the adjacency matrix, we get:

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8
x_1	1	1	0	0	0	0	0	1
x_2	1	1	1	0	0	0	0	0
x_3	0	1	1	1	0	0	0	0
x_4	0	0	1	1	1	0	0	0
x_5	0	0	0	1	1	1	0	0
x_6	0	0	0	0	1	1	1	0
x_7	0	0	0	0	0	1	1	1
x_8	1	0	0	0	0	0	1	1

If we weigh all of the edges, then we have the edge weighted adjacency matrix:

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8
x_1	1	0.9	0	0	0	0	0	0.3
x_2	0.9	2	1	0	0	0	0	0
x_3	0	1	1.5	0.8	0	0	0	0
x_4	0	0	0.8	1.1	0.9	0	0	0
x_5	0	0	0	0.9	1.3	0.7	0	0
x_6	0	0	0	0	0.7	1.1	0.9	0
x_7	0	0	0	0	0	0.9	0.9	1
x_8	0.3	0	0	0	0	0	1	1

Let this edge weighted adjacency matrix be referred to as Θ, and $p(x_1, \ldots, x_8) = N(\mu, \Theta^{-1})$

Conditional Independence via Markov Blanket

For the example above, which variables can we condition on to let X_1 be independent of the rest?

- **Local Markov property:** A variable is conditionally independent of all other variables given its neighbors
- Markov blanket of a node: **all of its neighbors**. (different from Markov blanket in Bayesian networks.) In our example, we would pick x_2 and x_8 since they neighbor x_1.
- A random variable is independent of all the other variables in the network, given its **Markov blanket**

$$X_1 \perp (X_3, \cdots, X_7) | (X_2, X_8)$$

Conditional Independence via pairwise Markov property

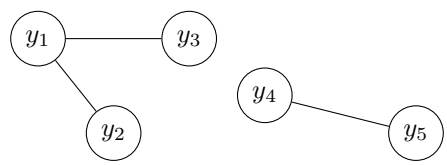
Pairwise Markov Property: Any two non-adjacent variables are conditionally independent given all other variables.

- In the above example, x_1 and x_3 are conditionally independent given all other variables, since they are non-adjacent.

Precision Matrix gives Graph structures

$$\begin{array}{c} y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \end{array} \begin{pmatrix} 1.3 & 0.3 & 0.8 & 0 & 0 \\ 0.3 & 1.0 & 0 & 0 & 0 \\ 0.8 & 0 & 1.2 & 0 & 0 \\ 0 & 0 & 0 & 1.5 & 1.1 \\ 0 & 0 & 0 & 1.1 & 0.9 \end{pmatrix}$$

The matrix represents a Gaussian graphical model encoded by Θ. It gives the following graph:



This aligns with the global markov property that any two subsets of variables are conditionally independent given a separating subset.

Covariance Matrix gives disconnected subgraphs

If we compute $\Theta^{-1} = \Sigma$ from the above matrix, we get:

$$\begin{array}{c} y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \end{array} \begin{pmatrix} 1.5 & -0.4 & -1.0 & 0 & 0 \\ -0.4 & 1.1 & 0.3 & 0 & 0 \\ -1.0 & 0.3 & 1.5 & 0 & 0 \\ 0 & 0 & 0 & 6.4 & -7.9 \\ 0 & 0 & 0 & -7.9 & 10.7 \end{pmatrix}$$

- The inverse of a block-diagonal matrix is also block-diagonal.
- Connected components can be found in both Θ and Σ

GGM Inference: Marginal Distribution

Going back to an example from before:

- How can we infer $p(x_1)$ from $p(x_1, \cdots, x_8)$

$$p(x_1, \ldots, x_8) = N(\mu, \Sigma)$$

$$\mu = \begin{bmatrix} \mu_1 \\ \vdots \\ \mu_8 \end{bmatrix} \qquad \Sigma = \begin{bmatrix} \sigma_1^2 & \cdots & \sigma_{18} \\ \vdots & \ddots & \vdots \\ \sigma_{18} & \cdots & \sigma_8^2 \end{bmatrix}$$

- What is $p(x_1)$?

$$N(\mu_1, \sigma_1^2)$$

- What is $p(x_1, x_2)$?

$$p(x_1, x_2) = N\left(\begin{bmatrix}\mu_1\\ \mu_2\end{bmatrix}, \begin{bmatrix}\sigma_1^2 & \sigma_{12}\\ \sigma_{12} & \sigma_2^2\end{bmatrix}\right)$$

- What is $p(x_1|x_2)$?

$$p(x_1|x_2) = \frac{p(x_1, x_2)}{p(x_2)}$$

Conditional Distribution is Gaussian

- Given the model

$$p(x_1, \ldots, x_8) = N(\mu, \Theta^{-1})$$

- One can derive a **conditional distribution** of a single random variable given **all other variables**

$$\begin{aligned}&p(x_1|x_2, \ldots, x_8)\\ &= N\left(\mu_1 + (x_{2:8} - \mu_{2:8})^T \Sigma_{2:8}^{-1}\Sigma_{2:8,1}, \sigma_{1,1}^2 - \Sigma_{1,2:8}^T \Sigma_{2:8}^{-1}\Sigma_{2:8,1}\right)\end{aligned}$$

Compare this with the regression model

$$p(y|X) = N(\beta_0 + X\beta, \sigma^2)$$

- The μ_1 in the conditional distribution and β_0 in the regression model are both prior means.
- The $(x_{2:8} - \mu_{2:8})^T$ maps to the X.
- The $\Sigma_{2:8}^{-1}\Sigma_{2:8,1}$ maps to the β

GGM Learning with MLE

Given samples $X^1, \ldots, X^N$, where each $X^i = [x_{i1}, \cdots, x_{iK}]$, for N samples and K variables

- Question: what is the MLE of Θ in $X \sim N(0, \Theta^{-1})$ (data is centered)
- Question: what to do if we want the graph to be sparse?

MLE for GGMs

- $X^i \sim N(0, \Sigma)$ with $\Theta = \Sigma^{-1}$
- MVN PDF:

$$\begin{aligned}f(x) &= (2\pi)^{-\frac{K}{2}} \det(\Sigma)^{-\frac{1}{2}} \exp\left(-\frac{1}{2}x^T\Sigma^{-1}x\right)\\ &\propto \det(\Theta)^{\frac{1}{2}} \exp\left(-\frac{1}{2}x^T\Theta x\right)\end{aligned}$$

- Log-likelihood:

$$\begin{aligned}l(\Theta) &= \frac{1}{N}\sum_{i=1}^{N} \log f(x^i)\\ &= \frac{1}{2}\log\det(\Theta) - \frac{1}{2N}\sum_{i=1}^{N}(x^i)^T\Theta x^i\\ &= \frac{1}{2}\log\det(\Theta) - \frac{1}{2}\mathrm{Tr}(S\Theta)\end{aligned}$$

- Note that $S = \frac{1}{2N}\sum_{i=1}^{N} x^i (x^i)^T$, the sample covariance.

Maximum Likelihood for GGM:

$$\arg\min_{\Theta} -\frac{1}{2}\log\det(\Theta) + \frac{1}{2}\mathrm{Tr}(S\Theta)$$

Solution: $\Theta^* = S^{-1}$.

Summary

- Gaussian graphical models
  - Undirected probabilistic graphical models
  - Continuous variables
- Gaussian!
  - Inference and learning are efficient. This is not true for undirected probabilistic graphical models for discrete variables